



Practica 7

[Practica7_2026.pdf](#)

Ejercicio 1:

1. ¿Qué es un sistema de tipos y cuál es su principal función?

-Un sistema de tipos es un conjunto de reglas que usa un lenguaje para estructurar y organizar sus tipos. El objetivo principal es escribir programas seguros, conocer los sistemas de tipos nos permite conocer mejor los aspectos semanticos de los lenguajes

2. Definir y contrastar las definiciones de un sistema de tipos fuerte y débil (probablemente en la bibliografía se encuentren dos definiciones posibles. Volcar ambas en la respuesta). Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

-Se dice que el sistema de tipos es fuerte cuando especifica restricciones sobre como las operaciones que involucran valores diferentes pueden operarse. Lo contrario se define en un sistema debil de tipos (definicion generica de fuertemente tipado).

-Lenguajes con sistemas de tipos fuertes:

-Python → dinamicamente tipado, fuertemente tipado porque no permite conversiones implicitas entre tipos incompatibles (sumar string e int) y asegura seguridad de tipos en ejecucion

-Java → estaticamente tipado, fuertemente tipado porque exige declaraciones de tipos y prohíbe operaciones entre tipos incompatibles sin casting explicito, garantizando tambien seguridad de tipos en la JVM

-Lenguajes con sistemas de tipos debiles:

-JavaScript → dinamicamente tipado, es debilmente tipado porque hace conversiones implicitas ("10" + 5 resulta en "105" o "10" - 5 que

da 5), lo que puede llevar a resultados inesperados

-C → estaticamente tipado, es debilmente tipado porque permite conversiones implicitas (sumar un int y float) y operaciones que violan la seguridad de tipos, interpretando un int como un char mediante punteros

3. Además de la clasificación anterior, también es posible caracterizar el tipado como estático o dinámico. ¿Qué significa esto? Ejemplificar con al menos 2 lenguajes para cada uno de ellos y justificar.

-Tipado estatico → Ligaduras en compilacion, se puede exigir:

-Se pueden utilizar tipos de datos predefinidos

-Todas las variables se declaran con un tipo asociado

-Todas las operaciones se especifican indicando los tipos de los operandos requeridos y el tipo del resultado

-Tipado dinamico → Ligadura en ejecucion, provoca mas comprobaciones en tiempo de ejecucion

Ejercicio 2: Tipos de datos:

1. Dar una definición de tipo de dato.

-Un tipo de dato es un conjunto de valores y un conjunto de operaciones asociadas al tipo para manejar esos valores

2. ¿Qué es un tipo predefinido elemental? Dar ejemplos.

-Un tipo de dato predefinido refleja el comportamiento del HW subyacente y es una abstraccion de el. Los elementales/escalares son tipos de datos predefinidos indivisibles, lo que quiere decir que no se pueden descomponer a partir de otros.

-Ejemplos → int, float, char, boolean

3. ¿Qué es un tipo definido por el usuario? Dar ejemplos.

-Los lenguajes de programacion permiten al programador definir agrupaciones de objetos de datos elementales y de forma recursiva, agrupaciones de agregados. Esto se logra mediante constructores, lo que se conoce como tipos de datos definidos por el usuario o nestas agrupaciones de datos elementales.

-Ejemplos → arrays, register, listas

Ejercicio 3: Tipos compuestos:

1. **Dar una breve definición de: producto cartesiano (en la bibliografía puede aparecer también como product type), correspondencia finita, uniones (en la bibliografía puede aparecer también como sum type) y tipos recursivos.**

-Producto cartesiano: Combinación de n conjuntos de tipos variados, lo que permite producir estructuras como registros en Pascal o structs en C, donde se agrupan datos de diferentes tipos en una sola entidad.

-Correspondencia finita: Función que mapea un conjunto finito de valores de un tipo de dominio DT (un entero) a valores de un tipo de dominio RT, accesibles a través de un índice. Esto se aplica a estructuras como listas indexadas, vectores, arreglos, matrices, que permiten organizar datos de manera estructurada y acceder a ellos mediante índices

-Union y Union discriminada: La unión de uno o más tipos representa la disyunción de los tipos dados, con campos mutuamente excluyentes que no pueden tener valores al mismo tiempo, permitiendo manipular diferentes tipos en distintos momentos de la ejecución. Esta estructura requiere chequeo dinámico, ya que no se puede asegurar en tiempo de compilación que variante o tipo adquiere una variable. Union discriminada mejora la unión al agregar un descriptor enumerativo que indica cuál de los campos tiene valor, brindando mayor seguridad al manipular el elemento según el discriminante, este descriptor puede omitirse si se quiere.

2. **Identificar a qué clase de tipo de datos pertenecen los siguientes extractos de código. En algunos casos puede corresponder más de una:**

Java <pre>class Persona { String nombre; String apellido; int edad; }</pre>	C <pre>typedef struct _nodoLista { void *dato; struct _nodoLista *siguiente } nodoLista; typedef struct _lista { int cantidad; nodoLista *primero } Lista;</pre>	C <pre>union codigo { int numero; char id; };</pre>
Ruby <pre>hash = { uno: 1, dos: 2, tres: 3, cuatro: 4 }</pre>	PHP <pre>function doble(\$x) { return 2 * \$x; }</pre>	Python <pre>tuple = ('physics', 'chemistry', 1997, 2000)</pre>

Haskell <pre>data ArbolBinarioInt = Nil Nodo int (ArbolBinarioInt dato) (ArbolBinarioInt dato)</pre> Ayuda para interpretar: 'ArbolBinarioInt' es un tipo de dato que puede ser Nil ("vacío") o un Nodo con un dato número entero (int) junto a un árbol como hijo izquierdo y otro árbol como hijo derecho	Haskell <pre>data Color = Rojo Verde Azul</pre> Ayuda para interpretar: 'Color' es un tipo de dato que puede ser Rojo, Verde o Azul.	
---	--	--

- Java → (clase Persona) Producto cartesiano
- C → Producto cartesiano en el struct, Recursion en lista
- C → Union
- Ruby → Correspondencia finita
- PHP → Correspondencia finita
- Python → Product cartesiano, y elementos de Correspondencia finita
- Haskell → Recursion y Union
- Haskell → Union

Ejercicio 4: Mutabilidad/Inmutabilidad

1. Definir mutabilidad e inmutabilidad respecto a un dato. Dar ejemplos en al menos 2 lenguajes. TIP: indagar sobre los tipos de datos que ofrece Python y sobre la operación #freeze en los objetos de Ruby.

-Mutabilidad e inmutabilidad: Se refieren a la capacidad o no de un dato de ser modificado despues de su creacion. Un dato mutable es aquel que puede cambiar y un dato inmutable es aquel que no puede cambiar

-Python → Ofrece datos inmujtables como enteros, strings y tuplas y datos mutables como listas, conjuntos y diccionarios

-Ruby → Ofrece tipos de datos mutables por defecto, a menos que utilices el metodo freeze que congela a un objeto y no se puede modificar

2. Dado el siguiente código:

```
a = Dato.new(1)
```

```
a = Dato.new(2)
```

¿Se puede afirmar entonces que el objeto "Dato.new(1)" es mutable?

Justificar la respuesta sea por afirmativa o por la negativa.

-No se puede afirmar que el objeto Dato.new(1) es mutable ya que no intenta modificar el estado del objeto Dato.new(1), simplemente se reasigna la variable a un nuevo objeto llamado Dato.new(2). Por lo que, solo esta reasignando y no cambiandolo, la reasignacion no implica mutabilidad ya que no cambia el estado interno de un objeto original

Ejercicio 5: Manejo de punteros:

1. ¿Permite C tomar el l-valor de las variables? Ejemplificar.

-Si, el lenguaje C permite tomar el l-valor y permite modificar el contenido de esa ubicacion mediante asignaciones yo peraciones. Esto se puede hacer medainte punteros

```

#include <stdio.h>

int main() {
    int x = 10;           // 'x' es un l-value, tiene una dirección de memoria
    int *ptr;             // Puntero para almacenar la dirección de 'x'

    ptr = &x;             // Tomamos el l-value de 'x' (su dirección) y lo asignamos a
    'ptr'                 // 'ptr'
    printf("Valor de x: %d\n", x); // Imprime 10
    printf("Dirección de x: %p\n", (void*)ptr); // Imprime la dirección de memoria de
    'x'

    *ptr = 20;            // Modificamos el contenido de la dirección de 'x' a través
    del puntero
    printf("Nuevo valor de x: %d\n", x); // Imprime 20

    return 0;
}

```

2. ¿Qué problemas existen en el manejo de punteros? Ejemplificar.

-Violación de tipos → Ocurre cuando un puntero se usa para acceder datos a datos de un tipo diferente al esperado, llevando a comportamiento indefinido o errores

```

int x = 10;
float *ptr = (float*)&x;
printf("%f\n", *ptr) // Acceso indebido, interpreta i
nt como float

```

-Referencias sueltas → Es un puntero que contiene una dirección de una variable dinámica que fue desalocada, si luego se usa se produce error

```

int *p = malloc(sizeof(int));
*p = 5;
free(p) // Liberamos P
printf("%d\n", *p) // Error referencia suelta

```

-Punteros no inicializados → Es un puntero que contiene una dirección aleatoria, lo que puede provocar accesos descontrolados a memoria si se usa sin inicialización previa

-Punteros y uniones discriminadas → Uso de punteros con uniones discriminadas puede permitir accesos indebidos si no se verifica correctamente el tipo de datos almacenado, accediendo a campos incorrectos

-Alias → Cuando dos o mas punteros comparten un alias (apuntan a la misma direccion de memoria) una modificacion de uno afecta a los demas, lo que puede causar efectos secundarios inesperados

Ejercicio 6: TAD:

1. ¿Qué características debe cumplir una unidad para que sea un TAD?

-Proveer mecanismos de expresion:

-Expresar tipos intrinsecos o definir tipos nuevos

-Asociar los tipos definidos con construcciones del lenguaje

-Definir reglas de resolucion:

-Equivalencia de tipos → dos valores tienen el mismo tipo?

-Compatibilidad de tipos → puedo usar el tipo en X contexto?

-Inferencia de tipos → cual tipo se deduce del contexto?

-Mientras mas flexible el lenguaje, mas complejo el sistema

2. Dar algunos ejemplos de TAD en lenguajes tales como ADA, Java, Python, entre otros.

-Modula-2: modulo

-Ada: paquete

-C++, Python, Java: clase

Ada, Java, Python y Haskell implementan TAD para estructuras como pilas, colas, conjuntos y listas, utilizando mecanismos como paquetes, interfaces, clases y tipos algebraicos